

# KRaft Dynamic Quorum State: How a Controller Knows Its Identity, Raft State, and Membership

## The core mental model

A useful way to understand a modern KRaft controller is to separate four different questions:

```
meta.properties
  = WHO AM I, AND WHICH KAFKA CLUSTER DOES THIS STORAGE BELONG TO?

quorum-state
  = WHAT LOCAL RAFT / ELECTION STATE MUST I REMEMBER ACROSS RESTARTS?

KRaft metadata log + snapshots
  = WHAT REPLICATED CLUSTER STATE DO I KNOW?

VotersRecord
  = WHO IS THE CURRENT DYNAMIC CONTROLLER VOTER SET?

controller.quorum.bootstrap.servers
  = WHERE CAN I INITIALLY CONNECT TO DISCOVER THE QUORUM?
```

That is essentially the model in the supplied design note, and it is a good abstraction for reasoning about KRaft startup and disaster recovery. The important refinement is that these objects do **not** have equal authority and do not describe the same kind of state.

`meta.properties` and `quorum-state` are local persistent state. The metadata log is replicated consensus state. `VotersRecord` is a KRaft control record carried by that replicated state.

`controller.quorum.bootstrap.servers`, by contrast, is only configuration used for discovery.

1

In a dynamic quorum, the distinction becomes particularly important because controller membership is no longer defined by a static configuration such as:

```
controller.quorum.voters=101@c101.dc1.example.com:
9093,102@c102.dc1.example.com:9093,103@c103.dc1.example.com:
9093,201@c201.dc2.example.com:9093,202@c202.dc2.example.com:9093
```

Instead, a dynamically configured controller typically has discovery endpoints such as:

```
controller.quorum.bootstrap.servers=c101.dc1.example.com:
9093,c102.dc1.example.com:9093,c103.dc1.example.com:
9093,c201.dc2.example.com:9093,c202.dc2.example.com:9093
```

The former encodes both voter IDs and addresses and therefore defines a static voter set. The latter contains only network endpoints and does **not** define who is entitled to vote. Apache Kafka's current KRaft documentation explicitly says that a dynamic quorum must not use

`controller.quorum.voters`; the actual quorum membership is managed dynamically, while `controller.quorum.bootstrap.servers` serves a role analogous to client `bootstrap.servers`.

2

A compact way of expressing the difference is therefore:

#### STATIC KRaft

```
controller.quorum.voters
  = node.id@hostname:port

  → membership + addressing
```

#### DYNAMIC KRaft

```
controller.quorum.bootstrap.servers
  = hostname:port

  → discovery only
```

#### Replicated VotersRecord

```
= node.id + directory.id + controller endpoints + version information

→ actual dynamic voter membership
```

KIP-853 is the architectural change behind this model. It moves the controller voter set into the KRaft replicated state so that membership changes can themselves be managed through the consensus protocol rather than requiring the same static voter configuration on every process. 3

## One important terminology correction

It is tempting to say:

"`VotersRecord` is the source of truth."

That is broadly right, but it should be understood as:

**the voter state reconstructed from the KRaft snapshot and metadata log is the replicated source of truth for dynamic membership.**

This wording matters because snapshots can contain the effective voter set, older log segments may already have been deleted, and a replica may temporarily encounter an **uncommitted VotersRecord** while a membership change is in progress. KIP-853 specifies that replicas update their current in-memory voter set when they read such a control record, but an uncommitted voter-set change can still be truncated. A membership operation is complete only when the required record has been committed under the appropriate quorum rules. <sup>4</sup>

So the most precise mental model is:



## Storage identity and meta.properties

`meta.properties` belongs to the local storage of a Kafka process. In KRaft mode it ties that storage to a Kafka cluster and a node identity. A representative current KRaft file looks conceptually like:

```

version=1
cluster.id=q1Sh-9_ISia_zwGINzRvyQ
node.id=201
directory.id=AbCdEfGhIjKlMnOpQrStUv
  
```

The three identity fields answer different questions:

| Field                     | Meaning                                                    |
|---------------------------|------------------------------------------------------------|
| <code>cluster.id</code>   | Which Kafka cluster this formatted storage belongs to      |
| <code>node.id</code>      | Which Kafka process identity the storage was formatted for |
| <code>directory.id</code> | Which particular storage directory instance this is        |

Apache Kafka's KRaft formatting process writes metadata associated with the cluster and storage directory, and KIP-853 relies specifically on the directory ID as part of controller voter identity. The directory ID is generated for a formatted log directory and persisted so that Kafka can distinguish a surviving replica from freshly created storage that happens to reuse the same numerical node ID. <sup>5</sup>

That distinction gives us a much stronger identity model than:

```
controller identity = node.id
```

For dynamic controller membership, the useful conceptual identity is:

```
controller replica identity
=
(node.id, directory.id)
```

For example:

```
old C201

node.id      = 201
directory.id = AAAA
```

and:

```
replacement C201

node.id      = 201
directory.id = BBBB
```

are not the same controller replica from the perspective of dynamic quorum membership:

```
(201, AAAA) != (201, BBBB)
```

KIP-853 explicitly uses this distinction for disk replacement. A controller with the same node ID but a different directory ID is treated as replacement storage, not silently accepted as the original voter. <sup>6</sup>

This protects against an especially dangerous recovery scenario. Suppose C201 was a voter:

```
Voter:
  node.id      = 201
  directory.id = AAAA
```

The storage is destroyed and an administrator formats a completely new metadata directory while keeping:

```
node.id=201
```

The new storage receives:

```
directory.id = BBBB
```

Without a storage incarnation identifier, Kafka would have difficulty distinguishing:

```
"the original replica 201 restarted"
```

from:

```
"a brand-new replica happens to be using node ID 201"
```

The directory ID makes that distinction explicit. KIP-853's disk-replacement workflow describes the old voter and the replacement as different `(node ID, directory ID)` tuples. <sup>7</sup>

This is also why removing a dynamic controller requires more than the numeric controller ID. Apache's `kafka-metadata-quorum.sh remove-controller` operation requires the controller ID together with its directory ID, so an operator identifies the exact voter incarnation that should be removed. <sup>5</sup>

`node.id` is not discovered only from `meta.properties`

There is an implementation nuance worth making explicit.

It would be slightly misleading to describe startup as:

```
Kafka reads meta.properties
      ↓
therefore it learns its node.id
```

The process also has a configured `node.id`. During KRaft server initialization, Kafka loads the local `meta.properties` information and validates it against the configured node identity and cluster/storage state. Current Kafka source performs consistency checks rather than treating arbitrary local metadata as permission to change the server's configured identity. <sup>8</sup>

A more accurate model is therefore:

```
server configuration
  node.id=201
    |
    v
  startup validation
    ^
    |
  meta.properties
    node.id=201
    cluster.id=CLUSTER-A
    directory.id=UUID201
```

In other words:

```
meta.properties
  = persistent proof of how this storage was formatted

server configuration
  = runtime configuration of the process

startup
  = verify that those identities are consistent
```

That distinction becomes very important during DR. Simply changing `node.id` in a configuration file does not magically transform existing KRaft storage into a different controller replica.

## Local Raft persistence and `quorum-state`

`quorum-state` solves a completely different problem.

It is not the Kafka cluster's membership database. It is persistent **local Raft state** associated with a KRaft partition, including election information that must survive process restarts. KIP-853 describes Kafka's persisted `QuorumStateData` and changes its schema for the newer KRaft protocol version.

9

Conceptually, its purpose is:

```
quorum-state

"what Raft/election state must this replica remember locally
so that a process restart does not erase protocol history?"
```

For `kraft.version=1`, the persisted structure includes fields corresponding to:

```
LeaderId
LeaderEpoch
VotedId
VotedDirectoryId
```

while older fields such as `CurrentVoters` are removed. KIP-853 explicitly removes `CurrentVoters` from `quorum-state` because voter membership is now represented through the KRaft log and snapshots instead. <sup>10</sup>

That separation is fundamental:

```
quorum-state
  ≠ current dynamic membership
```

Instead:

```
quorum-state
  = local persistent election / Raft protocol state
```

and:

```
snapshot + metadata log
  = replicated state, including voter membership
```

Raft requires important election state to persist because consensus safety depends on nodes not forgetting protocol decisions simply because their process restarted. In the underlying Raft model, elections and replicated-log decisions are governed by majority rules, and persistent election state prevents a reboot from erasing information needed to preserve those rules. <sup>11</sup>

The user-friendly statement:

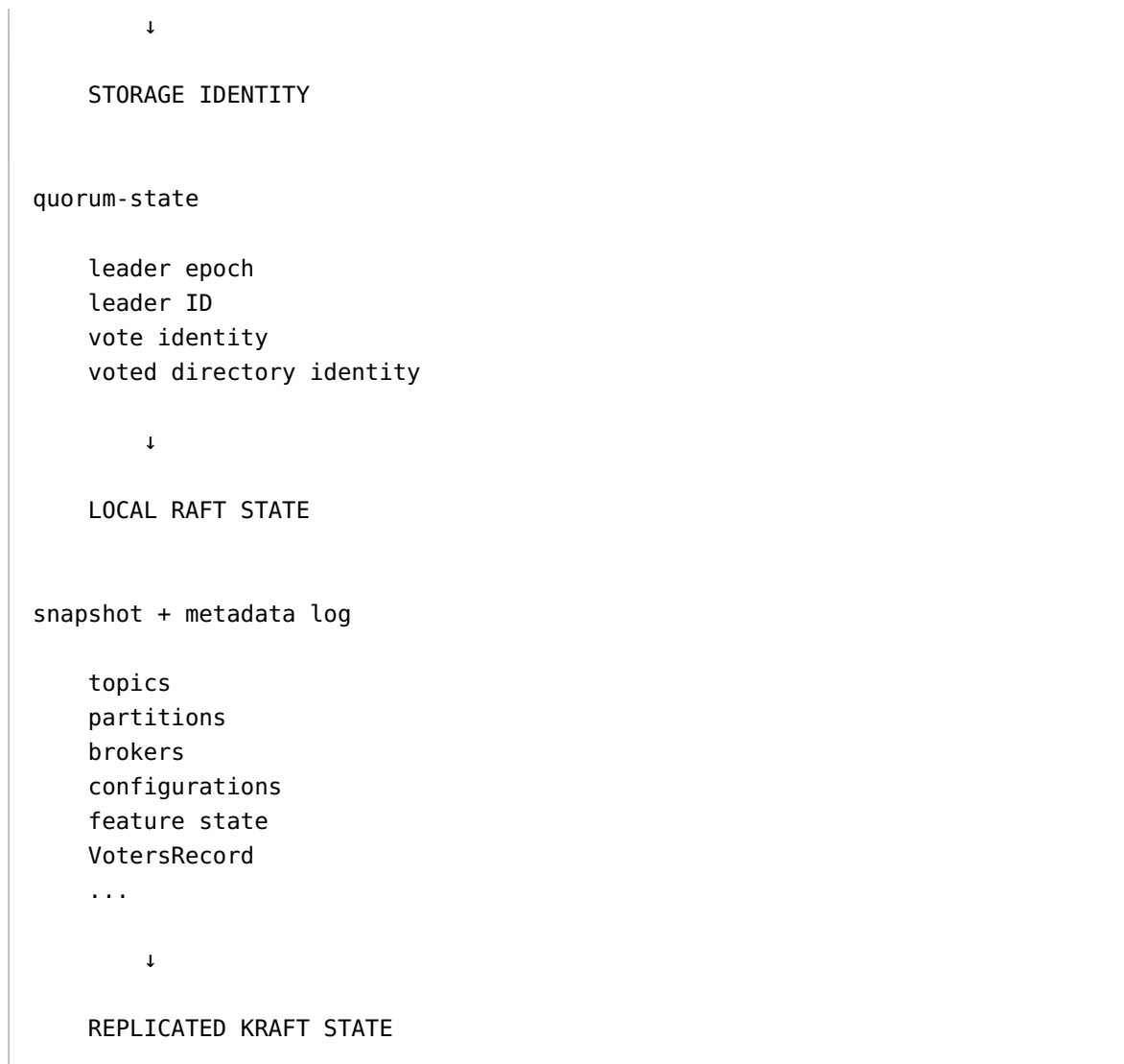
```
quorum-state = "what was my Raft state?"
```

is therefore useful, provided we do **not** interpret it as a second authoritative copy of the quorum definition. `filecite{turn0file0}`

A useful separation is:

```
meta.properties

  node.id
  cluster.id
  directory.id
```



### Why `CurrentVoters` disappearing matters

Under a static quorum, the controller set came from:

```
controller.quorum.voters=...
```

and Kafka could maintain local voter-related state against a membership definition that was effectively external and fixed.

Dynamic quorum changes the ownership of that information.

With KIP-853:

```
old model<br/><br/>configuration<br/>|
```



```

      v
controller.quorum.voters
      |
      v
fixed voter membership

```

becomes:

```

dynamic model

KRaft consensus
      |
      v
VotersRecord
      |
      v
replicated voter membership

```

That is why removing `CurrentVoters` from `quorum-state` is more than a file-format cleanup. Architecturally, it demonstrates that local election persistence is no longer supposed to be the place from which a node derives authoritative current membership. KIP-853 deliberately moves that responsibility to replicated quorum state. <sup>12</sup>

## Dynamic membership in `VotersRecord` and snapshots

`VotersRecord` is the central new control record for dynamic KRaft membership.

KIP-853 defines it as a record containing the current complete voter set. Each voter entry includes information such as:

```

VoterId
VoterDirectoryId
Endpoints
KRaftVersionFeature

```

and an endpoint contains fields including:

```

Name
Host
Port

```

The precise KIP schema includes the full voter list rather than representing membership merely as an incremental statement such as “add node 203.” When a membership change happens, a new voter state is recorded. <sup>13</sup>

For the example topology, we can visualize it as:

## VotersRecord

### Voters:

#### Voter:

```
VoterId          = 101
VoterDirectoryId = UUID101
Endpoints:
  CONTROLLER = c101.dc1.example.com:9093
```

#### Voter:

```
VoterId          = 102
VoterDirectoryId = UUID102
Endpoints:
  CONTROLLER = c102.dc1.example.com:9093
```

#### Voter:

```
VoterId          = 103
VoterDirectoryId = UUID103
Endpoints:
  CONTROLLER = c103.dc1.example.com:9093
```

#### Voter:

```
VoterId          = 201
VoterDirectoryId = UUID201
Endpoints:
  CONTROLLER = c201.dc2.example.com:9093
```

#### Voter:

```
VoterId          = 202
VoterDirectoryId = UUID202
Endpoints:
  CONTROLLER = c202.dc2.example.com:9093
```

This is conceptually very different from the old configuration:

```
controller.quorum.voters=101@c101.dc1.example.com:9093,...
```

The old configuration says:

```
"these configured identities constitute the quorum"
```

whereas the dynamic design says:

```
"the replicated KRaft state defines the voter set;
this configuration only helps processes contact the cluster"
```

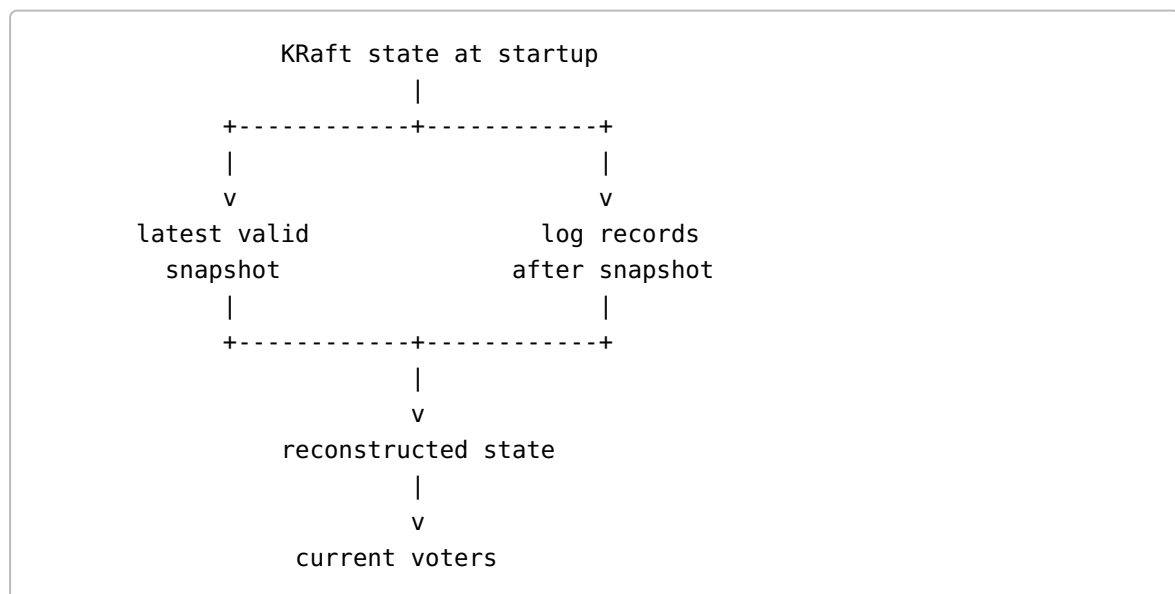
KIP-853 explicitly states that voter information and endpoints are stored in the log so they can be replicated to voters and observers. <sup>3</sup>

## The snapshot is part of the membership state

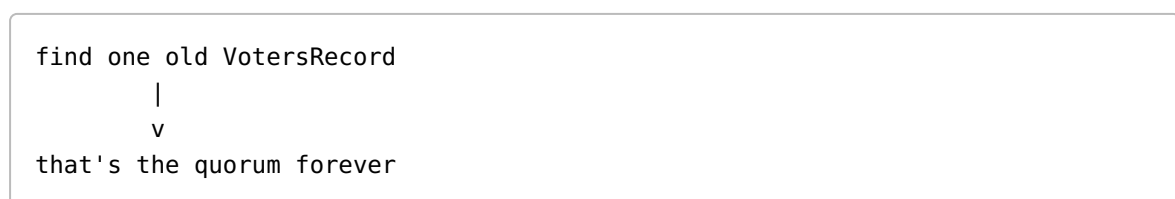
It is not correct to imagine that Kafka must always replay every historic `VotersRecord` beginning at offset zero.

KRaft uses snapshots. KIP-853 specifies KRaft-specific control records in snapshots, including the voter information needed to reconstruct quorum state at the snapshot point. Kafka can then replay records after that snapshot to obtain newer state. <sup>14</sup>

Therefore the disk representation is more accurately:



rather than:



Apache Kafka's formatting documentation also reflects this architecture. When initializing a dynamic controller quorum, the bootstrap snapshot can contain the initial `VotersRecord`; later quorum membership is subsequently represented by replicated records and snapshots. <sup>15</sup>

This matters enormously for offline DR analysis. Searching only log segment files for "the most recent visible `VotersRecord`" can be incomplete if the state you need is already represented in a snapshot. Apache provides tooling such as the metadata log decoder specifically for inspecting both metadata log data and snapshots. <sup>2</sup>

## Committed membership versus an in-flight membership change

There is another subtle point that matters during failures.

Suppose the current committed membership is:

VotersRecord A

101  
102  
103  
201  
202

and the leader begins adding:

203

The leader may append:

VotersRecord B

101  
102  
103  
201  
202  
203

KIP-853 specifies that replicas respond to the voter control record as it is read from the log. That means there can be a transient period in which the locally tracked “current voters” already reflects the new set even though the membership change has not yet become durably committed. If the operation cannot be committed, the uncommitted record can later disappear through Raft log truncation. <sup>16</sup>

That leads to an important DR distinction:

latest record physically present on disk  
≠ necessarily  
latest committed cluster state

and:

```
largest LogEndOffset
    ≠ necessarily
safest authoritative replica
```

A replica's log end can contain an uncommitted tail. The quorum's high watermark, by contrast, represents the committed boundary exposed by KRaft quorum status. Apache's quorum tooling reports both high-watermark information and per-replica log positions, which is precisely why those values should not be conflated. <sup>17</sup>

For DR, this is one of the most important conclusions in the entire model.

## Controller startup, discovery, and catch-up

Consider:

```
Controller: C201
node.id:    201
metadata:   /var/lib/kafka/kraft-metadata
```

with configuration conceptually containing:

```
process.roles=controller
node.id=201

metadata.log.dir=/var/lib/kafka/kraft-metadata

controller.quorum.bootstrap.servers=\
c101.dc1.example.com:9093,\
c102.dc1.example.com:9093,\
c103.dc1.example.com:9093,\
c201.dc2.example.com:9093,\
c202.dc2.example.com:9093
```

and local storage containing:

```
/var/lib/kafka/kraft-metadata/
  meta.properties
  __cluster_metadata-0/
  ...
  quorum-state
  log segments
  snapshots
```

The startup process is best understood as several logically separate recovery tasks. The exact implementation is intertwined inside Kafka's server and Raft initialization code, so the following should be read as a **conceptual dependency model**, not as a claim that every filesystem read occurs in exactly

this textual order. Kafka's current implementation loads and validates log-directory metadata, verifies node and cluster identity, initializes KRaft storage, and then brings up the Raft machinery around that persistent state. <sup>8</sup>

## Establish local storage identity

Kafka validates the formatted storage.

For C201:

```
meta.properties

cluster.id    = CLUSTER-A
node.id       = 201
directory.id  = UUID201
```

Conceptually:

```
QUESTION

"WHAT STORAGE IS THIS?"

      |
      v

+-----+
| meta.properties |
+-----+
| cluster = A     |
| node   = 201    |
| dir    = UUID201|
+-----+
```

The configured process identity must be compatible with the local storage identity; Kafka's startup code checks those values rather than blindly adopting mismatched metadata. <sup>18</sup>

## Restore persistent Raft election state

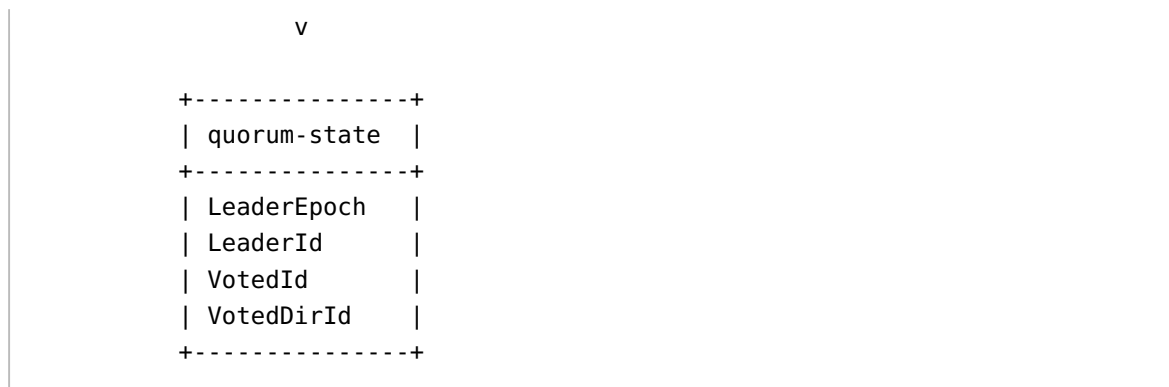
Kafka restores the Raft-local information represented by `quorum-state`.

Conceptually:

```
QUESTION

"WHAT MUST I REMEMBER
ABOUT RAFT/ELECTION?"

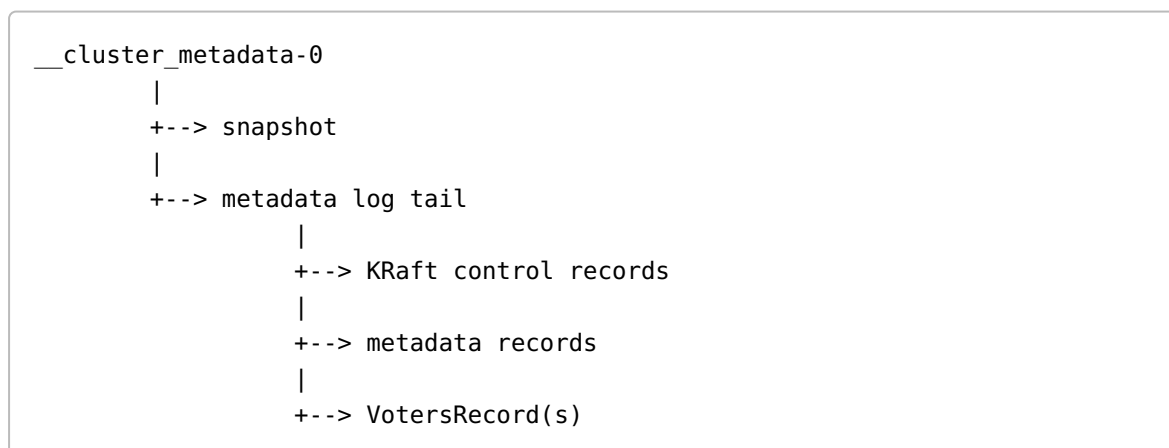
      |
```



This state is about continuity of the local consensus participant, not about defining the whole cluster's dynamic voter membership. <sup>9</sup>

## Reconstruct local KRaft state

The replica has some locally persisted view of the metadata partition:

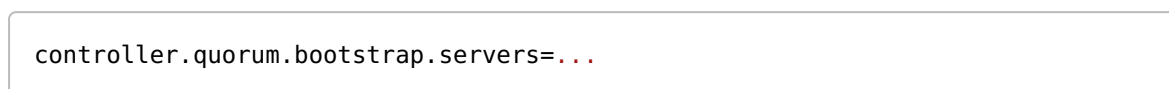


The snapshot plus subsequent records gives the local replica a reconstructed state at its current log position, including whatever voter state is represented there. <sup>19</sup>

At this point it is essential to use the word **local**. A controller that has been offline for a week may have perfectly valid local KRaft state that is nevertheless stale relative to the active quorum.

## Find the active quorum and leader

This is where:



comes into the architecture.

KIP-853 defines these addresses as endpoints through which brokers, observers, and newly joining controllers can discover the quorum leader. A process can contact listed endpoints and learn quorum information through KRaft replication rather than treating the bootstrap list itself as membership. <sup>13</sup>

So:

```
controller.quorum.bootstrap.servers
```

```
c101:9093
```

```
c102:9093
```

```
c103:9093
```

```
c201:9093
```

```
c202:9093
```

```
|
```

```
v
```

```
"try these addresses to reach the KRaft quorum"
```

```
|
```

```
v
```

```
discover leader / communicate with quorum
```

```
|
```

```
v
```

```
Fetch / FetchSnapshot
```

```
|
```

```
v
```

```
learn replicated metadata and current voter state
```

The current Kafka documentation notes that the bootstrap list does not have to contain every controller, although listing multiple usable endpoints improves discovery robustness. <sup>2</sup>

The crucial semantic distinction remains:

```
bootstrap server present in configuration
```

```
DOES NOT MEAN
```

```
that server is currently a voter
```

and:

```
voter absent from bootstrap list
```

```
DOES NOT NECESSARILY MEAN
```

```
that it is absent from the dynamic voter set
```

Membership and discovery are independent concepts. <sup>1</sup>



## Catch up stale metadata

Suppose C201 was stopped while the active quorum changed from:

old:

101  
102  
103  
201  
202

to:

new:

101  
102  
103  
201  
203

C201's disk can initially contain older replicated state:

local C201 state

Voters:

101 102 103 201 202

LogEndOffset:

older

while the active quorum contains:

active quorum state

Voters:

101 102 103 201 203

LogEndOffset:

newer

Once C201 communicates with the functioning quorum and fetches missing metadata or a newer snapshot, it can move toward the quorum's newer replicated state. The newer voter record supersedes the stale membership represented in its old local log. KIP-853 explicitly designs voter information so followers and observers learn it through the replicated log and snapshots. <sup>3</sup>

Conceptually:

```
C201 restart

local disk says:
101 102 103 201 202
    |
    v
contact active quorum
    |
    v
discover leader
    |
    v
Fetch / FetchSnapshot
    |
    v
receive newer KRaft state
    |
    v
VotersRecord says:
101 102 103 201 203
```

The incorrect mental model would be:

```
C201 sees C202 in its old local copy
    |
    v
therefore C202 must still be a voter
```

Dynamic KRaft does not work that way.

### An important failure caveat

There is, however, an equally dangerous opposite oversimplification:

“The node can always use `controller.quorum.bootstrap.servers` to recover from any stale membership problem.”

That is not true.

Bootstrap addresses are useful while there is a functioning quorum or at least a reachable participant capable of guiding discovery. They do not override persisted consensus membership. KIP-853 intentionally makes membership a replicated property, not a local configuration override. <sup>3</sup>

This distinction becomes critical after loss of a quorum majority. In fact, a later Apache Kafka proposal, KIP-1347, exists specifically because certain dynamic-quorum endpoint and majority-loss situations are difficult or impossible to repair safely using normal configuration alone. As of the researched Kafka

state in 2026, KIP-1347 is still described as a proposal under discussion rather than a generally available recovery mechanism. <sup>20</sup>

So:

```
bootstrap servers
  = discovery hints under normal quorum operation
```

```
bootstrap servers
  ≠ emergency authority override
```

```
bootstrap servers
  ≠ replacement for consensus
```

```
bootstrap servers
  ≠ proof of current voter membership
```

## Membership changes and storage replacement

Dynamic quorum membership is not simply a matter of editing a configuration file and restarting servers.

KIP-853 makes membership change part of the replicated consensus protocol. Apache's current operational procedure is to provision a controller, allow it to catch up, and then perform the controller-add operation so that the quorum records the new voter state. <sup>17</sup>

Consider the existing five-voter set:

```
101
102
103
201
202
```

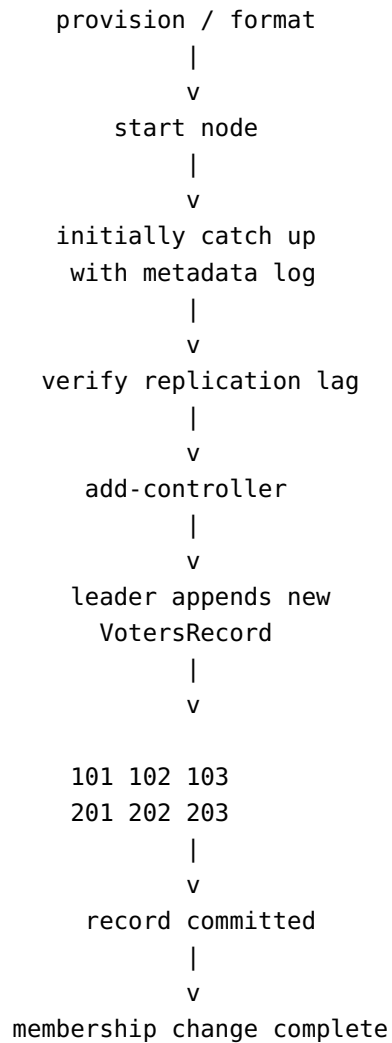
and a new C203:

```
node.id      = 203
directory.id = UUID203
```

A simplified lifecycle is:

```
C203
```

```
|
v
```

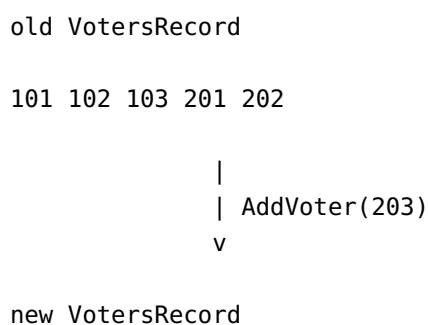


Apache Kafka's KRaft documentation explicitly recommends monitoring the new controller's replication state before adding it as a voter. KIP-853 additionally requires the new replica to catch up to the leader before the add-voter change is appended, reducing the risk that a newly admitted voter is immediately too far behind to participate effectively. <sup>21</sup>

## Membership change is itself a consensus operation

The new voter set does not become durable merely because some process wrote a new configuration.

Instead:



```
101 102 103 201 202 203
```

is replicated through KRaft.

KIP-853's safety design handles changes one at a time and applies quorum rules that preserve the required relationship between the old and new voter sets while the transition is committed. <sup>22</sup>

That gives membership much stronger semantics than:

```
edit config on five machines
restart
hope everyone agrees
```

Instead, membership itself is subject to the same replicated-log machinery that Kafka uses for controller consensus.

### Why an even-sized temporary quorum may appear

If C203 is being added as a replacement before an old voter is removed, the set can temporarily become:

```
101 102 103 201 202 203
```

That is six voters.

For six voters the majority threshold is four, whereas the original five-voter set requires three. An even-numbered quorum does not provide additional failure tolerance compared with the preceding odd-sized quorum: both five and six voters tolerate at most two unavailable voters while retaining a majority. Kafka's operational documentation consequently recommends the conventional odd-sized controller quorum model such as three or five controllers. <sup>23</sup>

For a replacement operation, the temporary six-voter state is therefore best understood as a transition:

```
5 voters
  |
  | add replacement
  v
6 voters
  |
  | remove old voter
  v
5 voters
```

rather than an objective in itself.

## Replacing a disk under the same node ID

Now take the C201 example in more detail.

Before disk failure:

C201-old

```
node.id      = 201
directory.id = AAAA
```

and the voter set contains:

(201, AAAA)

After replacing and reformatting the metadata storage:

C201-new

```
node.id      = 201
directory.id = BBBB
```

The new process represents:

(201, BBBB)

It is a different replica identity.

KIP-853's recovery example explicitly uses the directory ID to distinguish these two incarnations. The replacement can catch up as a non-voting participant and then be admitted, after which the old voter tuple can be removed. <sup>6</sup>

Conceptually:

old membership:

```
201 + AAAA
  |
  | disk permanently lost
  v
```

replacement storage:

201 + BBBB

```

    |
    | catch up
    v

temporarily:

201 + AAAA
201 + BBBB

    |
    | remove old voter identity
    v

final:

201 + BBBB

```

This illustrates why the directory ID is not cosmetic metadata. It allows the consensus layer to distinguish an original voter from storage that has been recreated under the same `node.id`.

## Disaster-recovery implications for the C201/C202 scenario

This architecture has a direct consequence for the DR scenario described in the supplied material.

filecite turn0file0

Assume the controller placement is:

```

DC1
  C101
  C102
  C103

DC2
  C201
  C202

```

with current voters:

```

101 102 103 201 202

```

A five-voter quorum requires three votes for a majority. Kafka documents the standard `2N+1` relationship: a quorum of five can tolerate two unavailable controllers while maintaining consensus.

23

That means the two datacenter-failure cases are asymmetric.

Lose DC2:

```
survivors = C101 C102 C103  
          = 3/5
```

majority remains  
→ quorum can continue

but:

Lose DC1:

```
survivors = C201 C202  
          = 2/5
```

majority lost  
→ quorum cannot continue normally

This is not specific to Kafka implementation quirks; it follows directly from majority consensus. <sup>11</sup>

## When a majority still exists

If at least three members of this five-voter quorum remain healthy, standard Kafka recovery semantics apply.

Apache's current controller-disk replacement guidance advises verifying the state of the remaining quorum and its replication before formatting and introducing replacement storage.

`kafka-metadata-quorum.sh describe --replication` can expose replication positions and lag, while `describe --status` reports information including the cluster ID, leader, leader epoch, high watermark, current voters, and voter directory IDs. <sup>24</sup>

Typical online inspection is conceptually:

```
kafka-metadata-quorum.sh \  
  --bootstrap-controller <controller-host:port> \  
  describe --status
```

and:

```
kafka-metadata-quorum.sh \  
  --bootstrap-controller <controller-host:port> \  
  describe --replication
```

The important information includes:



```
ClusterId
LeaderId
LeaderEpoch
HighWatermark

CurrentVoters:
  id
  directoryId
  endpoints

per-replica:
  LogEndOffset
  lag
  fetch/caught-up information
```

Apache exposes these fields specifically to allow operators to understand the state and synchronization of the controller quorum. <sup>25</sup>

With a live majority, the leader and high watermark give you a consensus-backed reference point. A failed controller can be rebuilt according to normal dynamic membership procedures.

## When DC1 is permanently lost

The difficult case is:

```
C101 = lost
C102 = lost
C103 = lost

C201 = survives
C202 = survives
```

Now only:

```
2 of 5
```

voters remain.

At this point there is no quorum that can elect a leader and advance consensus normally. More importantly, there is no longer a surviving majority from which you can trivially prove the complete latest committed metadata history. Apache's KIP-1347 discussion explicitly describes majority-loss recovery as a case for which Kafka does not currently have a generally safe automatic recovery solution; the proposal warns that committed metadata can have been lost with the missing majority and that an operator may not be able to determine authoritative state from the minority alone. <sup>20</sup>

This requires an important correction to the original DR formulation.

The statement:

"We will determine which `VotersRecord` and metadata offset C201/C202 have and select the safer one as the seed."

is a useful **forensic investigation step**, but it is not sufficient by itself to prove a safe reconstruction.

Why?

Imagine the leader was C101 shortly before DC1 disappeared.

A five-voter quorum needs three acknowledgements. This metadata record could have become committed on:

```
C101
C102
C103
```

without ever reaching:

```
C201
C202
```

before the entire datacenter was lost.

Therefore:

```
C201 latest local state
C202 latest local state
```

may both be behind a state that had already been committed by the original quorum.

That conclusion follows from the combination of the five-voter majority rule and the fact that three controllers were colocated in DC1. <sup>26</sup>

The surviving minority cannot infer:

```
"I have offset X, therefore X must be the latest committed
offset that ever existed"
```

because the missing majority may have committed metadata after that survivor's last successful fetch.

### **The highest surviving offset is not automatically safest**

There is a second failure mode in the opposite direction.

Suppose:

```
C201 LogEndOffset = 5000
C202 LogEndOffset = 4998
```

It would be dangerous to conclude:

```
C201 has the larger offset
      ↓
C201 must contain more committed state
      ↓
C201 is the authoritative seed
```

Raft logs can contain records beyond the high watermark. KIP-853 explicitly deals with membership records that can be read before they are committed and subsequently truncated if consensus does not complete. Therefore a longer surviving log can contain an uncommitted tail. <sup>27</sup>

The general relationship is:

```
LogEndOffset
  = how far this local log physically extends

HighWatermark
  = committed boundary known through quorum consensus
```

Thus:

```
larger LEO
  ≠ proof of greater committed authority
```

Apache's tooling reports high watermark separately from per-replica log positions for precisely this reason. <sup>2</sup>

For offline DR analysis, you therefore want to inspect substantially more than:

```
latest offset
```

You want to correlate:

```
cluster.id

node.id + directory.id

snapshot IDs / snapshot offsets
```

```
log segment ranges

leader epochs

quorum-state election information

VotersRecord history

last locally represented voter set

metadata record sequence

known committed/high-watermark information, when available
```

The metadata log decoder can inspect KRaft log segments and snapshots, while the online quorum tools expose consensus state when a quorum is still available. <sup>2</sup>

A practical offline inspection target is therefore:

```
C201 copy
|
+--> meta.properties
|
+--> quorum-state
|
+--> snapshot
|
+--> metadata log after snapshot
|
+--> all visible VotersRecord transitions
|
+--> final log / epoch information
```

```
C202 copy
|
+--> meta.properties
|
+--> quorum-state
|
+--> snapshot
|
+--> metadata log after snapshot
|
+--> all visible VotersRecord transitions
|
+--> final log / epoch information
```



Apache's current KRaft tooling includes `kafka-dump-log.sh --cluster-metadata-decoder` for decoding the metadata log and snapshots, which is the appropriate class of tool for understanding what is actually present in each surviving metadata directory. <sup>2</sup>

## What “safe seed” should mean

For this DR design, I would define the term very carefully:

SAFE SEED

not:

"the surviving node with the largest offset"

not:

"the node with the newest-looking VotersRecord"

not:

"the node whose quorum-state names the newest leader"

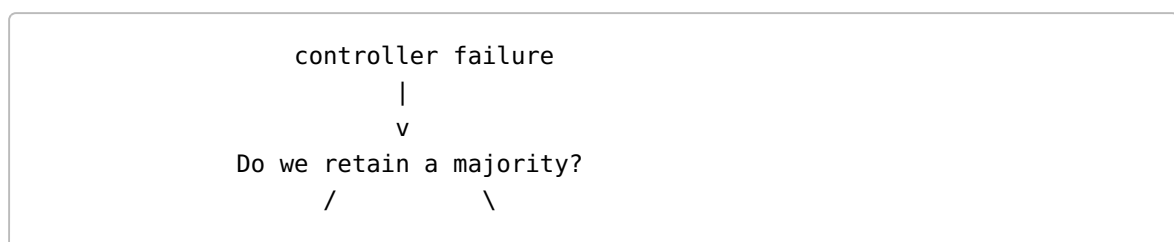
but:

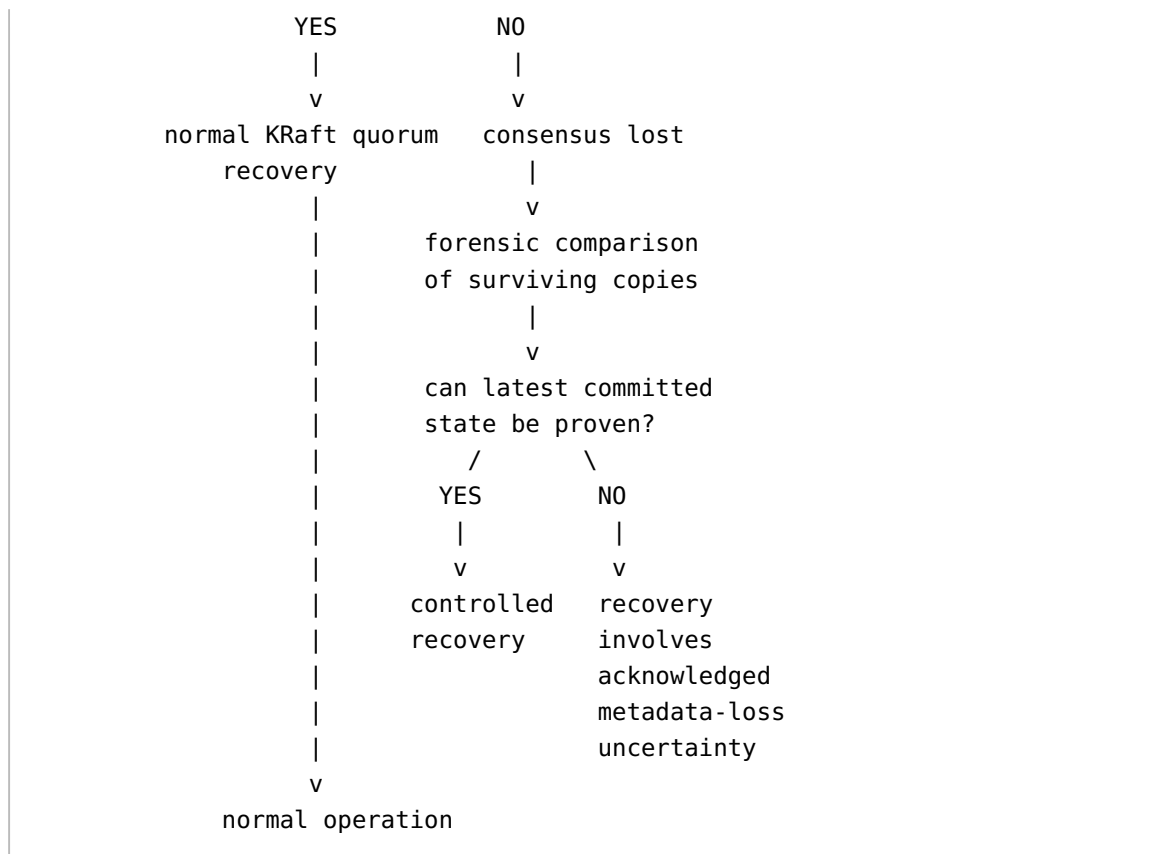
"a replica whose state can be demonstrated to contain every metadata record that was committed by the original quorum up to the recovery boundary"

With a surviving majority, consensus itself gives you a mechanism for establishing that committed boundary.

With only C201 and C202 surviving from a five-voter quorum, that proof may simply be unavailable. KIP-1347's majority-loss discussion is explicit about this class of uncertainty: re-bootstrap after permanent majority loss can imply metadata loss, and the surviving minority cannot in general know everything the lost majority had committed. <sup>20</sup>

That means the correct DR decision tree is closer to:





This is the key distinction between **recovering a failed controller while the quorum survives** and **reconstructing a KRaft cluster after permanently losing the quorum majority**. The former is a normal supported operational workflow; the latter is fundamentally a consensus-loss event and must not be presented as merely “pick the freshest disk and restart.” <sup>28</sup>

## Why the two-DC layout matters

There is also an architectural lesson in the original topology:

DC1: 3 voters  
DC2: 2 voters

This layout deliberately or implicitly chooses DC1 as the failure domain whose loss the quorum **cannot** tolerate:

loss of DC2  
→ 3/5 survive  
→ quorum survives

loss of DC1  
→ 2/5 survive  
→ quorum lost

No placement of an odd five-member majority across only two sites can make the quorum independently tolerate complete loss of **either** site: one of the two locations must contain fewer than three voters and the other must contain at least three. This follows from majority arithmetic. <sup>29</sup>

That topology question is separate from Kafka's data-replication topology for user topics. It specifically concerns availability and recoverability of the KRaft controller metadata quorum.

The complete operational model for the five-controller example can therefore be summarized as:

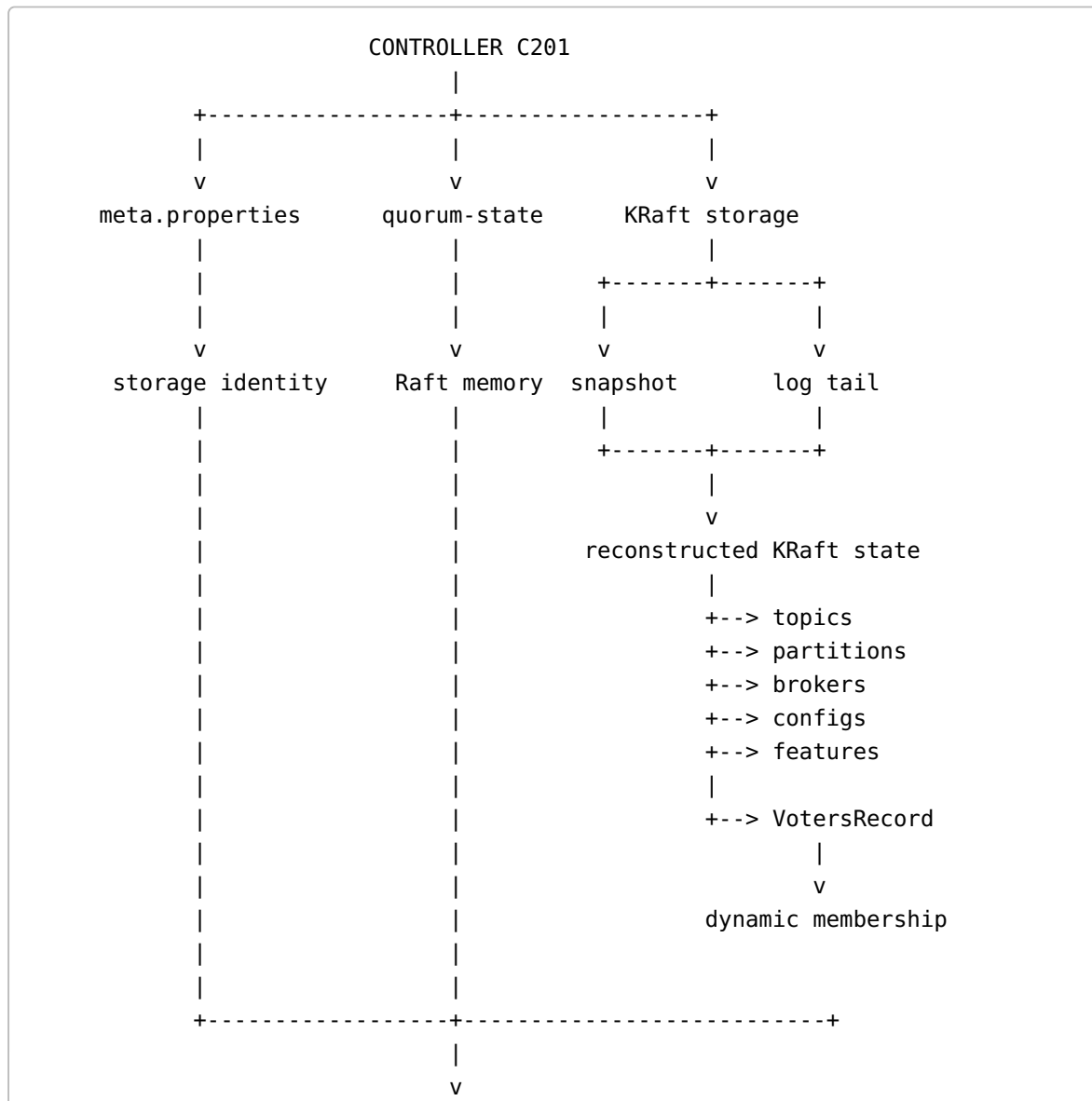
| Element                                          | Physical / logical location            | Primary purpose                                                                                 | Authority during DR                                         |
|--------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <code>meta.properties</code>                     | Local metadata storage                 | Binds storage to <code>cluster.id</code> , <code>node.id</code> , and <code>directory.id</code> | Identity evidence; not cluster membership authority         |
| <code>quorum-state</code>                        | Local KRaft metadata partition storage | Persists local Raft election state                                                              | Useful forensic/protocol state; not voter-set authority     |
| <code>VotersRecord</code>                        | KRaft metadata log and snapshots       | Describes dynamic voter membership                                                              | Replicated membership state; commitment status matters      |
| KRaft snapshot                                   | Metadata storage                       | Checkpoint of replicated metadata state, including quorum-related state                         | Essential part of reconstructed local cluster state         |
| Metadata log tail                                | <code>__cluster_metadata-0</code>      | Changes after the snapshot                                                                      | Must be interpreted together with snapshot and commit state |
| <code>controller.quorum.bootstrap.servers</code> | Server configuration                   | Initial quorum discovery                                                                        | Discovery only; cannot redefine membership                  |

| Element       | Physical / logical location   | Primary purpose                | Authority during DR                              |
|---------------|-------------------------------|--------------------------------|--------------------------------------------------|
| HighWatermark | Runtime KRaft consensus state | Boundary of committed metadata | Strongly relevant when quorum is operational     |
| LogEndOffset  | Per-replica log               | End of local physical log      | Does not by itself prove commitment or authority |

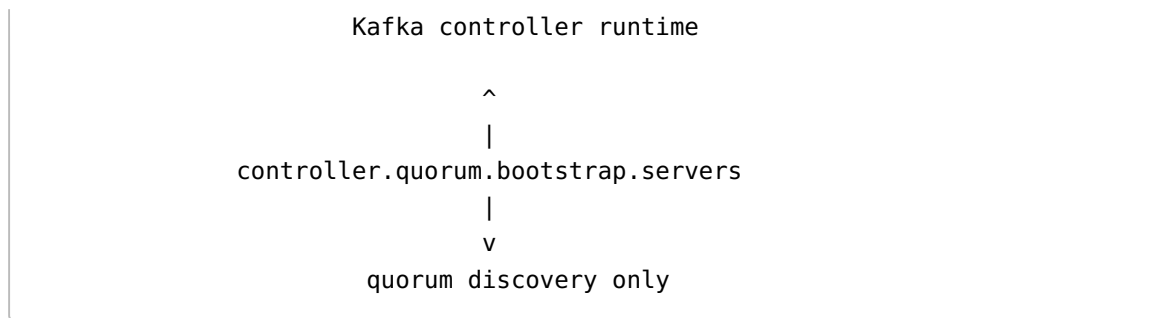
These roles are defined by current KRaft documentation and KIP-853's dynamic membership design.

30

The final mental model is therefore:







KIP-853 changes the most important part on the right-hand side of this diagram: controller voter membership is no longer fundamentally a static property supplied through `controller.quorum.voters`, nor is it carried as `CurrentVoters` in the newer `quorum-state` representation. It becomes replicated KRaft state represented through `VotersRecord` and snapshots.

12

For the planned DC1-loss DR procedure, the practical consequence is equally important: examining the `VotersRecord`, snapshots, epochs, `quorum-state`, and metadata offsets of C201 and C202 is exactly the right forensic direction, but **those observations alone do not make either survivor a provably safe seed after losing three of five voters**. A surviving minority may be missing metadata that was committed exclusively on the lost majority, while its own highest offsets may include uncommitted records. The first DR question must therefore be not merely “which survivor has the highest offset?” but “**can we prove the committed recovery boundary of the original quorum?**” If the answer is no, the operation has crossed from ordinary controller recovery into an acknowledged majority-loss recovery scenario with potential metadata-loss risk. 31

1 2 5 15 17 21 23 25 30 <https://kafka.apache.org/43/operations/kraft/>  
<https://kafka.apache.org/43/operations/kraft/>

3 4 6 7 9 10 12 13 14 16 19 22 27 <https://cwiki.apache.org/confluence/spaces/KAFKA/pages/217391519/KIP-853%2BKRaft%2BController%2BMembership%2BChanges>  
<https://cwiki.apache.org/confluence/spaces/KAFKA/pages/217391519/KIP-853%2BKRaft%2BController%2BMembership%2BChanges>

8 18 <https://github.com/apache/kafka/blob/trunk/core/src/main/scala/kafka/server/KafkaRaftServer.scala>  
<https://github.com/apache/kafka/blob/trunk/core/src/main/scala/kafka/server/KafkaRaftServer.scala>

11 26 <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>  
<https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>

20 31 <https://cwiki.apache.org/confluence/spaces/KAFKA/pages/421958889/KIP-1347%2BOverriding%2Bvoter%2Bset%2Bon%2Bstorage%2Bformatting>  
<https://cwiki.apache.org/confluence/spaces/KAFKA/pages/421958889/KIP-1347%2BOverriding%2Bvoter%2Bset%2Bon%2Bstorage%2Bformatting>

24 28 <https://kafka.apache.org/43/operations/hardware-and-os/>  
<https://kafka.apache.org/43/operations/hardware-and-os/>

29 <https://raft.github.io/index.html>  
<https://raft.github.io/index.html>